Guía rápida de SQL

Val Marcos
marcos.e.val01@gmail.com

Institución:

FCEyN - Universidad de Buenos Aires
Carrera: Lic. en Física

Última actualización:

24 de marzo de 2026

Índice

1. Modificaciones sobre tablas **5**

1.1. Creación de tablas (create table) 5

1.2. Eliminar tablas (drop table) 5

1.3. Información sobre tablas y columnas (sp_tables - sp_columns) 5

1.4. Insertar registros de una tabla (insert into) 6

1.5. Recuperar registros de una tabla (select) 6

1.6. Recuperar registros con condiciones (where) 6

1.7. Borrar registros (delete) 6

1.8. Actualizar registros (update) 7

1.9. Truncate table 7

1.10. Alias (as) 7

1.11. Crear tabla a partir de otra (select - into) 7

2. Tipos de datos **8**

2.1. Texto 8

2.2. Numéricos 9

2.3. Fechas 9

2.4. Valores null (is null) 10

2.5. Clave primaria (primary key) 11

2.6. Clave primaria compuesta (primary key II) 11

2.7. Campo con atributo Identity 11

2.7.1. ident_seed() 12

2.7.2. ident_incr() 12

2.7.3. identity_insert 13

2.8. Variable de usuario 13

3. Operadores **14**

3.1. Funciones para el manejo de cadenas (caracteres) 14
3.2. Funciones matemáticas 18
3.3. Funciones de uso de fechas y horas 19
3.4. Operadores lógicos 21

4. Consultas (Queries) 22

4.1. Ordenar registros (order by) 22
4.2. Búsqueda de patrones (like - not like) 22
4.3. Contar registros (count) 23
4.4. Contar registros (count_big) 24
4.5. Funciones de agrupamiento(sum - min - max - avg) 24
4.6. Agrupar registros (group by) 24
4.7. Seleccionar grupos (having) 25
4.8. Modificadores del group by (with rollup - with cube) 25
4.9. with cube 26
4.10. grouping 26
4.11. Registros duplicados (distinct) 27
4.12. Primeros registros (top - with ties) 27
4.13. Lenguaje de control de flujo (case) 28
4.14. Lenguaje de control de flujo II (if) 29

5. Restricciones y reglas 29

5.1. Información sobre restricciones (sp_helpconstraint) 29
5.2. default 30
5.3. check 30
5.4. Deshabilitar restricciones (with check - nocheck) 31
5.5. Restricción primary key 31
5.6. unique 31
5.7. Elimiar restricciones (drop) 32

5.8. Crear y asociar reglas (create rule - sp_bindrule) 32

5.9. Valores predeterminados (create default - sp_bindefault) 33

5.10. Desasociar y eliminar valores predeterminados (sp_unbindefault - drop) 34

5.11. Información sobre reglas (sp_help - sp_helpconstraint) 34

6. índices **35**

6.1. Crear índices 37

6.2. Regenerar índices 37

6.3. Eliminar índices 38

7. Combinaciones **38**

7.1. Combinaciones básicas (join - left/right join - full join) 38

7.2. Combinación cruzada (cross join) 40

7.3. Autocombinación 40

7.4. Combinaciones con update - delete 41

7.5. Combinación de más de dos tablas 41

8. Clave foránea (foreign key) y restricciones **41**

8.1. Restricciones foreign key en la misma tabla 42

8.2. Acciones 43

9. Subconsultas **44**

9.1. Subconsulta como valor único 44

9.2. Subconsulta con in 44

9.3. any - some - all 45

9.4. Subconsultas correlacionadas 46

9.5. Subconsultas con exist - not exist 46

9.6. Simil autocombinación 47

9.7. Subconsulta como tabla 47

9.8. Subconsulta con update - delete - insert 47

10. Vistas **48**

10.1. Vistas en Bases de Datos 48

10.2. Información sobre vistas 49

10.3. Encriptado (with encryption) 50

10.4. Eliminar una vista (drop view) 51

10.5. Vistas with check option 51

10.6. Modificar datos de tablas a traves de vistas 52

10.7. Modificar vistas (alter view) 52

11. OTROS **53**

11.1. Operadores relacionales 53

11.2. Comando go 53

Introducción

Según el estudio reciente [2], este método es efectivo.

Varios autores apoyan esta teoría [1, 2].

1. Modificaciones sobre tablas

1.1. Creación de tablas (create table)

Una base de datos almacena su información en tablas.

Una tabla es una estructura de datos que organiza los datos en columnas y filas; cada columna es un campo (o atributo) y cada fila, un registro. La intersección de una columna con una fila, contiene un dato específico, un solo valor. Las tablas forman parte de una base de datos.

Al crear una tabla debemos resolver qué campos (columnas) tendrá y que tipo de datos almacenarán cada uno de ellos, es decir, su estructura. La sintaxis básica y general para crear una tabla es la siguiente:

```
CREATE TABLE usuarios (
nombre VARCHAR(100),
...
email VARCHAR(255),
);
```

Con el comando **VARCHAR** lo que estamos haciendo es imponer que el campo `nombre` puede contener una cadena de hasta 100 caracteres. Analogamente para `email` tenemos hasta 255 caracteres.

1.2. Eliminar tablas (drop table)

```
drop table usuarios;
-- O si queremos asegurarnos que no salte un error
if object_id('usuarios') is not null
  drop table usuarios;
```

1.3. Información sobre tablas y columnas (sp_tables - sp_columns)

```
exec sp_tables@table_owner='dbo'

exec sp_columns usuarios
```

El parámetro `@table_owner='dbo'` indica que solo muestre las tablas de usuarios y no las que crea el SQL Server para administración interna.

1.4. Insertar registros de una tabla (insert into)

Al ingresar los datos de cada registro debe tenerse en cuenta la cantidad y el orden de los campos. Si tenemos una tabla que consta de dos únicos campos:

```
insert into usuarios (nombre, email)
values ('Mariano', 'mariano.p@gmail.com');
```

En este caso si nuestra tabla consta de más columnas el resto de campos en la filla ingresada va a quedar como valores "**null**" si no hay condiciones que nos lo impidan. También es posible agregar una fila entera en la tabla, para ello tenemos que respetar el orden de los campos y las caracteristicas de cada uno:

```
insert into usuarios
values ('Mariano', 'mariano.p@gmail.com', '25'); --Agragamos la tabla
   edad
```

1.5. Recuperar registros de una tabla (select)

Existen dos formas de recuperar resultados de una tabla:

- '*' para seleccionar todos los elementos
- 'nombre_columna1, ..., nombre_columna n ' si queremos tener n - columnas

```
select * from usuarios; --Si queremos todos los resultados
select titulo, precio from libros; --Si queremos algunas columnas
```

1.6. Recuperar registros con condiciones (where)

```
select nombre, clave from usuarios
where nombre = 'Marcelo';
```

1.7. Borrar registros (delete)

Si queremos eliminar los registros de una tabla, pero no la tabla utilizamos esta función. Nos permite eliminar registros de manera total o parcial usando la condición **where**.

```sql
delete from usuarios  --Si queremos borrar todos los registros

delete from usuarios  --borra los registros que cumplan la condicion
where nombre = 'Marcelo'
```

1.8. Actualizar registros (update)

Podemos actualizar uno o varios registros con el comando `update` a un valor nuevo designado por el comando `set`

```sql
update usuarios set equipo = 'RealMadrid' --Modifica toda la columna
   equipos

--Podemos cambiar algunos registros y combinar condiciones
update usuarios set nombre = 'Marcelo_Perez', equipo = 'RealMadrid'
where nombre = 'Marcelo';
```

1.9. Truncate table

Sirve para vaciar los registros de una tabla. La diferencia con `delete` es la velocidad ya que esta última elimina los registros uno a uno. Por su lado `drop table` permite eliminar una tabla por completo, mientras que `truncate table` la vacia.

```sql
truncate table usuarios;
```

1.10. Alias (as)

Es una manera de cambiar el nombre de los encabezados de cada columna a la hora de hacer la consulta, no es un cambio permanente.

```sql
select nombre as 'Nombre y apellido'
domicilio, telefono
from agenda;
```

Las comillas no son necesarias cuando no tenemos espacios en la palabra.

1.11. Crear tabla a partir de otra (select - into)

Podemos crear una tabla e insertar datos en ella en una sola sentencia consultando otra tabla (o varias) con esta sintaxis:

```
select CAMPOSNUEVATABLA
into NUEVATABLA
from TABLA
where CONDICION;
```

Podemos emplear **group by**, funciones de agrupamiento y **order by** en las consultas. También podemos emplear "**select** ... **into**" con combinaciones, para crear una tabla que contenga datos de 2 o más tablas.

2. Tipos de datos

2.1. Texto

Tipo de dato	Descripción
varchar(x)	Define una cadena de caracteres de longitud variable en la cual determinamos el máximo de caracteres con el argumento "x" que va entre paréntesis. Si se omite el argumento coloca 1 por defecto. Su rango va de 1 a 8000 caracteres.
char(x)	Define una cadena de longitud fija determinada por el argumento "x". Si se omite el argumento coloca 1 por defecto. Su rango es de 1 a 8000 caracteres. Si la longitud es invariable, es conveniente utilizar el tipo char; caso contrario, el tipo varchar. Ocupa tantos bytes como se definen con el argumento "x". "char" viene de character, que significa carácter en inglés.
text	Guarda datos binarios de longitud variable, puede contener hasta 2×10^9 caracteres. No admite argumento para especificar su longitud.
nvarchar(x)	Es similar a "varchar", excepto que permite almacenar caracteres Unicode, su rango va de 0 a 4000 caracteres porque se emplean 2 bytes por cada carácter.
nchar(x)	Es similar a "char" excepto que acepta caracteres Unicode, su rango va de 0 a 4000 caracteres porque se emplean 2 bytes por cada carácter.
ntext	Es similar a "text" excepto que permite almacenar caracteres Unicode, puede contener hasta 10^9 caracteres. No admite argumento para especificar su longitud.

2.2. Numéricos

Tipo de dato	Descripción
`integer` o **`int`**	Números enteros sin decimales. Ocupa 4 bytes de almacenamiento. Rango: aproximadamente $-2{,}15 \times 10^9$ a $2{,}15 \times 10^9$.
`tinyint`	Números enteros muy pequeños. Ocupa 1 byte de almacenamiento. Rango: 0 a 255.
`smallint`	Números enteros pequeños. Ocupa 2 bytes de almacenamiento. Rango: aproximadamente $-3{,}20 \times 10^4$ a $3{,}20 \times 10^4$.
`bigint`	Números enteros muy grandes. Ocupa 8 bytes de almacenamiento. Rango: aproximadamente $-9{,}00 \times 10^{18}$ a $9{,}00 \times 10^{18}$.
`decimal` o **`numeric`**	Números de precisión fija con decimales. Se define como "decimal(p,s)" donde "p" es la precisión (total de dígitos) y "s" es la escala (dígitos decimales).
`float`	Números de punto flotante de precisión variable. Se usa para números muy grandes o muy pequeños. Rango aproximado: $-1{,}79 \times 10^{38}$ a $1{,}79 \times 10^{38}$.
`real`	Números de punto flotante de precisión simple. Ocupa 4 bytes. Rango aproximado: $-3{,}40 \times 10^{38}$ a $3{,}40 \times 10^{38}$.
`money`	Tipo de dato para valores monetarios. Almacena valores de aproximadamente $-9{,}22 \times 10^{14}$ a $9{,}22 \times 10^{14}$. Ocupa 8 bytes.
`smallmoney`	Tipo de dato para valores monetarios más pequeños. Almacena valores de aproximadamente $-2{,}15 \times 10^5$ a $2{,}15 \times 10^5$. Ocupa 4 bytes.

2.3. Fechas

Tipo de dato	Descripción
`datetime`	Almacena fechas y horas combinadas. Rango: desde el 1 de enero de 1753 hasta el 31 de diciembre de 9999. Precisión de hasta 3.33 milisegundos. Ocupa 8 bytes de almacenamiento.
`smalldatetime`	Almacena fechas y horas combinadas con un rango más limitado. Rango: desde el 1 de enero de 1900 hasta el 6 de junio de 2079. Precisión de hasta 1 minuto. Ocupa 4 bytes de almacenamiento.

Las fechas se ingresan entre comillas simples. Para almacenar valores de tipo fecha se permiten como separadores "/", "-" y ".".

SQL Server reconoce varios formatos de entrada de datos de tipo fecha. Para establecer el orden de las partes de una fecha (día, mes y año) empleamos "`set dateformat`". El formato por defecto es "mdy". Los distintos formatos son:

- `mdy`: 4/15/96 (mes y día con 1 ó 2 dígitos y año con 2 ó 4 dígitos)
- `myd`: 4/96/15
- `dmy`: 15/4/1996
- `dym`: 15/96/4
- `ydm`: 96/15/4
- `ydm`: 1996/15/4

Para ingresar una fecha con formato "día-mes-año", tipeamos:

```
set dateformat dmy
```

2.4. Valores null (is null)

La palabra "null" significa "dato desconocido" o "valor inexistente". No es lo mismo que un valor "0", una cadena vacía o una cadena literal "null".

A veces, puede desconocerse o no existir el dato correspondiente a algún campo de un registro. En estos casos decimos que el campo puede contener valores nulos. En contraposición, tenemos campos que no pueden estar vacíos jamás. Para crear una tabla en la cual no admitimos valores nulos en sus campos usamos el comando `not null`:

```
create table libros(
titulo varchar(30) not null,
autor varchar(20) not null,
editorial varchar(15) null,
precio float
);
```

Cuando colocamos "null" estamos diciendo que admite valores nulos (caso del campo "editorial"); por defecto los campos permiten valores nulos (caso del campo "precio").

Si ingresamos los datos de un libro, para el cual aún no hemos definido el precio podemos colocar "null" para mostrar que no tiene precio. Notemos que null no debe ir entre comillas ya que no es una cadena de texto.

```
insert into libros (titulo, autor, editorial, precio)
values('El aleph', 'Borges', 'Emece', null)
```

2.5. Clave primaria (primary key)

Una clave primaria es un campo (o conjunto de campos) en los que se identifica un solo registro por fila.

```
create table usuarios(
 nombre varchar(20),
 clave varchar(10),
 primary key(nombre)
);
```

Cualquier campo (de cualquier tipo) puede ser clave primaria, debe cumplir como requisito, que sus valores no se repitan ni sean nulos. Por ello, al definir un campo como clave primaria, automáticamente SQL Server lo convierte a "not null".

Una tabla sólo puede tener una clave primaria, lo que no significa que esta pueda ser compuesta:

```
create table inscripciones(
 id_estudiante int,
 id_curso int,
 fecha date,
 primary key (id_estudiante, id_curso)
);
```

2.6. Clave primaria compuesta (primary key II)

Hay situaciones en las que no alcanza con definir un único campo como **primary key**. Si queremos ver los autos que entran y salen de un estacionamiento a una dada hora no podemos identificarlos unívocamente por su patente (porque el auto puede salir y entrar en una misma hora) o la hora de llegada (ya que varios autos pueden entrar a la vez), pero si podemos pedir que una combinación de estas dos sea nuestra nueva **primary key**.

```
create table vehiculos(
patente char(6) not null,
tipo char(1),--'a'=auto, 'm'=moto
horallegada datetime,
horasalida datetime,
primary key(patente,horallegada)
);
```

2.7. Campo con atributo Identity

Un campo numérico puede tener un atributo extra “identity”. Los valores de un campo con este atributo genera valores secuenciales que se inician en 1 y se incrementan en 1 automáticamente.

Se utiliza generalmente en campos correspondientes a códigos de identificación para generar valores únicos para cada nuevo registro que se inserta. Sólo puede haber un campo "identity" por tabla.
Para que un campo pueda establecerse como "identity", éste debe ser entero (también puede ser de un subtipo de entero o decimal con escala 0).

```
 create table libros(
codigo int identity,
titulo varchar(40) not null,
autor varchar(30),
editorial varchar(15),
precio float
);
```

Cuando un campo tiene el atributo "identity" no se puede ingresar valor para él, porque si usamos el comando **insert**, automáticamente toma el último valor como referencia, o 1 si es el primero. Uno puede indicar el lugar de comienzo del identado y el salto del mismo usando **identity**(start, step).

```
create table libros(
codigo int identity(100,2),
titulo varchar(40) not null,
autor varchar(30),
editorial varchar(15),
precio float
);
```

Este comando tiene algunas propiedades extra que son útiles:

2.7.1. ident_seed()

Retorna el valor de inicio del campo "identity" de la tabla que nombramos:

```
select ident_seed('libros');
```

2.7.2. ident_incr()

Retorna el valor de incremento del campo "identity" de la tabla nombrada:

```
select ident_incr('libros');
```

2.7.3. identity_insert

Permite ingresar un valor en un campo de identidad al activar la opción con el siguiente comando:

```
set identity_insert libros on;
```

Cuando `identity_insert` está en ON, las instrucciones **insert** deben explicitar un valor:

```
insert into libros (codigo,titulo)
 values (5,'Alicia en el pais de las maravillas');
```

Si no se coloca un valor para el campo de identidad, la sentencia no se ejecuta y aparece un mensaje de error.

2.8. Variable de usuario

Las variables nos permiten almacenar un valor y recuperarlo más adelante para emplearlos en otras sentencias.

Las variables de usuario son específicas de cada conexión y son liberadas automáticamente al abandonar la conexión. La sintaxis basica es:

```
declare @NOMBREVARIABLE TIPO

--Ejemplo con varias variables
declare @nombre varchar(20), @edad int
```

Una variable declarada existe dentro del entorno en que se declara; debemos declarar y emplear la variable en el mismo lote de sentencias, porque si declaramos una variable y luego, en otro bloque de sentencias pretendemos emplearla, dicha variable ya no existe.

Se utiliza **set** para darle un valor a la variable, de lo contrario esta adquiere un valor **null**

```
set @edad=45

--o con una consulta select
 select @nombre = autor from libros where titulo='Uno'
```

La utilidad de las variables consiste en que almacenan valores para utilizarlos en otras consultas.

Por ejemplo, queremos saber todos los datos del libro con mayor precio de la tabla "libros" de una librería. Para ello podemos emplear una variable para almacenar el precio más alto:

```
--Buscamos el precio mas alto
declare @mayorprecio decimal(5,2)
```

```sql
select @mayorprecio=max(precio) from libros;

go --para separar las sentencias

--Mostramos libros con mayor precio
select *from libros
where precio=@mayorprecio;
```

Es decir, declaramos la variable y guardamos en ella el precio más alto y luego, en otra sentencia, mostramos los datos de todos los libros cuyo precio es igual al valor de la variable.

3. Operadores

Los operadores son una forma simple de poder realizar cuentas entre campos y mostrarlos como una colmna o elemento separado (o añadirlo) a las tablas.

podemos hacer calculos entre numeros de la siguiente manera:

```sql
--Queremos una columna extra con el 90% de descuento
select titulo, precio,
precio * 0.9
from libros;

--Podemos actualizar una columna usando operaciones
update libros set precio = precio * 0.9
```

Tambien podemos concatenar texto usando el operador "+":

```sql
select titulo+'-'+autor+'-'+editorial
from libros;
```

El guion "(-)" es solamente estético, puede remplazarse por un espacio.

3.1. Funciones para el manejo de cadenas (caracteres)

Una función es un conjunto de sentencias que operan como una unidad lógica.

Una función tiene un nombre, retorna un parámetro de salida y opcionalmente acepta parámetros de entrada. Las funciones de SQL Server no pueden ser modificadas y algunas de estas para trabajar con caracteres son:

Función y Descripción	Ejemplo
substring(cadena, inicio, longitud): Devuelve una parte de la cadena desde la posición inicial con la longitud especificada.	`select substring('Buenas tardes',8,6)` `-- Retorna 'tardes'`

str(numero, longitud, decimales): Convierte números a cadenas. Longitud y decimales son opcionales. Además si la longitud es menor que la parte entera, devuelve asteriscos.	`select str(123.456,7,3)`
	`-- Retorna '123.456'`
	`select str(123.456,2,3)`
	`-- Retorna '**'`

stuff(cadena1, inicio, cantidad, cadena2): Inserta cadena2 en cadena1 desde la posición inicio, reemplazando cantidad de caracteres.	`select stuff('abcde',3,2,'opqrs')` `-- Retorna 'abopqrse'`
len(cadena): Retorna la longitud de la cadena.	`select len('Hola')` `-- Retorna 4`
char(x): Retorna el carácter ASCII del entero x.	`select char(65)` `-- Retorna 'A'`
left(cadena, longitud): Retorna longitud caracteres comenzando desde la izquierda.	`select left('buenos dias',8)` `-- Retorna 'buenos d'`
right(cadena, longitud): Retorna longitud caracteres comenzando desde la derecha.	`select right('buenos dias',8)` `-- Retorna 'nos dias'`

Cuadro 1: Funciones de cadena en SQL (Parte 1)

Función y Descripción	Ejemplo
lower(cadena): Convierte la cadena a minúsculas.	`select lower('HOLA ESTUDIAnte')` `-- Retorna 'hola estudiante'`
upper(cadena): Convierte la cadena a mayúsculas.	`select upper('HOLA ESTUDIAnte')` `-- Retorna 'HOLA ESTUDIANTE'`
`ltrim`(cadena): Elimina espacios en blanco del lado izquierdo.	`select ltrim('    Hola    ')` `-- Retorna 'Hola    '`
`rtrim`(cadena): Elimina espacios en blanco del lado derecho.	`select rtrim('   Hola  ')` `-- Retorna '   Hola'`
replace(cadena, cadenareemplazo, cadenareemplazar): Reemplaza todas las ocurrencias de una subcadena por otra.	`select replace('xxx.sqlserverya.com','x', 'w')` `-- Retorna 'www.sqlserverya.com'`
`reverse`(cadena): Invierte el orden de los caracteres de la cadena.	`select reverse('Hola')` `-- Retorna 'aloH'`

`patindex`(patron, cadena): Retorna la posición del patrón en la cadena. Retorna 0 si no lo encuentra.	`select patindex('%Luis%', 'Jorge Luis Borges')`
	`-- Retorna 7`
	`select patindex('%ar%', 'Jorge Luis Borges')`
	`-- Retorna 0`

Cuadro 2: Funciones de cadena en SQL (Parte 2)

Función y Descripción	Ejemplo

`charindex`(subcadena, cadena, inicio): Retorna la posición de la subcadena comenzando desde inicio.	`select charindex('or', 'Jorge Luis Borges',5)`
	`-- Retorna 13`
	`select charindex('or', 'Jorge Luis Borges')`
	`-- Retorna 2`

`replicate`(cadena, cantidad): Repite una cadena la cantidad de veces especificada.	`select replicate('Hola', 3)` `-- Retorna 'HolaHolaHola'`
`space`(cantidad): Retorna una cadena de espacios de longitud indicada.	`select 'Hola'+space(1)+'que tal'` `-- Retorna 'Hola que tal'`

3.2. Funciones matemáticas

Las funciones matemáticas realizan operaciones con expresiones numéricas y retornan un resultado, operan con tipos de datos numéricos. Microsoft SQL Server tiene algunas funciones para trabajar con números. Aquí presentamos algunas.

Función y Descripción	Ejemplo
`abs`(x): Retorna el valor absoluto del argumento x.	`select abs(-20)` `-- Retorna 20`
`ceiling`(x): Redondea hacia arriba el argumento x.	`select ceiling(12.34)` `-- Retorna 13`
`floor`(x): Redondea hacia abajo el argumento x.	`select floor(12.34)` `-- Retorna 12`

`%`: Devuelve el resto de una división.	`select 10 % 3`
	`-- Retorna 1`
	`select 10 % 2`
	`-- Retorna 0`

`power`(x, y): Retorna x elevado a la y potencia.	`select power(2, 3)` `-- Retorna 8`

`round`(numero, longitud): Redondea un número a la longitud especificada.	`select round(123.456, 1)`
	`-- Retorna 123.400`
	`select round(123.456, -1)`
	`-- Retorna 120.000`

Función y Descripción	Ejemplo

`sign`(x): Retorna 1 si x es positivo, -1 si es negativo, 0 si es cero.	`select sign(15)`
	`-- Retorna 1`
	`select sign(-8)`
	`-- Retorna -1`
	`select sign(0)`
	`-- Retorna 0`

`square`(x): Retorna el cuadrado del argumento x.	`select square(3)` `-- Retorna 9`
`sqrt`(x): Retorna la raíz cuadrada del argumento x.	`select sqrt(9)` `-- Retorna 3`

3.3. Funciones de uso de fechas y horas

Funcion y Descripcion	Ejemplo
`getdate()`: Retorna la fecha y hora actuales.	`select getdate()` `-- Retorna fecha y hora actual`

`datepart`(partedefecha, fecha): Retorna la parte especifica de una fecha (año, mes, dia, etc.).	`select datepart(month, getdate())`
	`-- Retorna el numero del mes actual`
	`select datepart(day, getdate())`
	`-- Retorna el dia actual`
	`select datepart(hour, getdate())`
	`-- Retorna la hora actual`

Funcion y Descripcion	Ejemplo

`datename`(partedefecha, fecha): Retorna el nombre de una parte especifica de una fecha.	`select datename(month, getdate())`
	`-- Retorna el nombre del mes actual`
	`select datename(day, getdate())`
	`-- Retorna el nombre del dia actual`

`dateadd`(partedelafecha, numero, fecha): Agrega un intervalo a la fecha especificada.	`select dateadd(day, 3, '1980/11/02')`
	`-- Retorna '1980/11/05'`
	`select dateadd(month, 3, '1980/11/02')`
	`-- Retorna '1981/02/02'`
	`select dateadd(hour, 2, '1980/11/02')`
	`-- Retorna '1980/11/02 2:00:00'`

`datediff`(partedelafecha, fecha1, fecha2): Calcula el intervalo entre dos fechas.	`select datediff(day, '2005/10/28', '2006/10/28')`
	`-- Retorna 365`
	`select datediff(month, '2005/10/28', '2006/11/29')`
	`-- Retorna 13`

`day`(fecha): Retorna el dia de la fecha especificada.	`select day(getdate())` `-- Retorna el dia actual`
`month`(fecha): Retorna el mes de la fecha especificada.	`select month(getdate())` `-- Retorna el mes actual`
`year`(fecha): Retorna el año de la fecha especificada.	`select year(getdate())` `-- Retorna el ano actual`

3.4. Operadores lógicos

Operador	Ejemplo
AND	`SELECT * FROM libros` `WHERE precio > 100 AND editorial = 'Penguin'`
OR	`SELECT * FROM libros` `WHERE lanzamiento < 2000 OR lanzamiento > 2020`
NOT	`SELECT * FROM libros` `WHERE NOT editorial = 'Anaya'`
BETWEEN	`SELECT * FROM libros` `WHERE precio BETWEEN 50 AND 200`
IN[1]	`SELECT * FROM libros` `WHERE autor NOT IN ('Borges','Paenza')`

( ): Paréntesis para agrupar condiciones y definir orden de evaluación.	`SELECT * FROM libros`
	`WHERE (precio > 100 AND editorial = 'Penguin')`
	`OR (lanzamiento > 2020 AND genero = 'Ciencia Ficcion')`

[1]Recuerde: siempre que sea posible, emplee condiciones de búsqueda positivas ("in"), evite las negativas ("not in") porque con ellas se evalúan todos los registros y esto hace mas lenta la recuperación de los datos.

4. Consultas (Queries)

4.1. Ordenar registros (order by)

Podemos ordenar el resultado de un **select** para que los registros se muestren ordenados por algún campo, para ello usamos la cláusula **order by**.

```
select * from libros
order by titulo;

--Podemos ordenarlos por varios campos en orden
select * from libros
order by titulo, precio;
```

También podemos llamar a los campos por su orden en la tabla.

```
--Ordenamos por precio
select * from libros
order by 2;
```

Por último, podemos ordenar los campos en orden ascendente o descendente:

```
--Ordenamos por precio
select * from libros
order by titulo asc, precio desc;
```

4.2. Búsqueda de patrones (like - not like)

Podemos comparar trozos de cadenas de caracteres para realizar consultas. Para recuperar todos los registros cuyo autor contenga la cadena "Borges"debemos tipear:

```
select * from libros
where autor like '%Borges%'
```

El símbolo " %" (porcentaje) reemplaza cualquier cantidad de caracteres (incluyendo ningún caracter). Es un caracter comodín. **like** y **not like** son operadores de comparación que señalan igualdad o diferencia.

Por otro lado, el guión bajo " _" reemplaza un carácter, es otro caracter comodín. Por ejemplo, queremos ver los libros de "Lewis Carroll"pero no recordamos si se escribe Çarroll.º Çarrolt", entonces tipeamos esta condición

```
select * from libros
where autor like '%Carrol_'
```

Otro carácter comodín es "[ ]" reemplaza cualquier carácter contenido en el conjunto especificado dentro de los corchetes.

```
select * from libros
where editorial ...

--Ejemplos

... like '[a-cf-i]%'--busca cadenas que comiencen con a,b,c,f,g,h o i;
... like '[-acfi]%'--busca cadenas que comiencen con -,a,c,f o i;
... like 'A[_]9%'--busca cadenas que comiencen con 'A_9';
... like 'A[nm]%'--busca cadenas que comiencen con 'An' o 'Am'.
```

El tercer caracter comodín es "[$\hat{]}$" reemplaza cualquier caracter NO presente en el conjunto especificado dentro de los corchetes.

```
--Editoriales que no empiecan con P ni N
select * from libros
where editorial like '[^PN]%'
```

Para búsquedas de caracteres comodines como literales, debe incluirlo dentro de corchetes, por ejemplo, si busca:

```
select * from libros
where editorial ...

--Ejemplos

... like '%[%]%'--busca cadenas que contengan el signo '%';
... like '%[_]%'--busca cadenas que contengan el signo '_';
... like '%[[]%'--busca cadenas que contengan el signo '[';
```

4.3. Contar registros (count)

La función `count()` cuenta la cantidad de registros o filas de una tabla, incluyendo los que tienen valor nulo.

```
--Contamos todas las filas
select count(*)
from libros

--Contamos las filas en un campo de una tabla
select count(precio)
from libros
```

4.4. Contar registros (count_big)

Es el análogo de **count()** con la diferencia de que `count_big()` devuelve un "bigint" (mayor rango que un integer).

```
select count_big(precio)
from libros;
```

4.5. Funciones de agrupamiento(sum - min - max - avg)

Al igual que la función **count()** existen más funciones que nos permiten agrupar registros. Se pueden usar en una instrucción **select** y combinarlas con la cláusula **group by**. Todas estas funciones retornan **null** si ningún registro cumple con la condición del **where**, excepto **count** que en tal caso retorna cero.

Funcion	Ejemplo
sum(): Suma de valores	`select sum(cantidad)` `from libros;`
min(): Valor minimo	`select min(precio)` `from libros;`
max(): Valor maximo	`select max(precio)` `from libros;`

avg(): Promedio de valores	`select avg(precio)`
	`from libros`
	`where titulo like '%PHP%';`

4.6. Agrupar registros (group by)

Una forma para poder agrupar uno (o varios) campos a la hora de hacer consultas es con el comando **group by**. Esto tambien permite hacer consultas con otras funciones de agrupamiento

como **count**() y otras operaciones en grupos más chicos "por bloques".

```
select count(titulo)
from libros
group by editorial;
```

4.7. Seleccionar grupos (having)

Podemos seleccionar o rechazar ciertos elementos de un grupo con el comando **having**. Se utiliza **having**, seguido de la condición de búsqueda, para seleccionar ciertas filas retornadas por la cláusula **group by**.

```
select editorial, count(*) from libros
group by editorial
having editorial<>'Planeta'

--Ejemplo con where
--select editorial, count(*) from libros
--where editorial<>'Planeta'
--group by editorial;
```

Los comandos **where** y **having** parecen similares, pero tienen sus diferencias. El primero selecciona todos los registros rechazando los de editorial "Planeta" y luego los agrupa para contarlos. El segundo selecciona todos los registros, los agrupa para contarlos y finalmente rechaza fila con la cuenta correspondiente a la editorial "Planeta".

No debemos confundir la cláusula **where** con la cláusula **having**; la primera establece condiciones para la selección de registros de un **select**; la segunda establece condiciones para la selección de registros de una salida **group by**.

4.8. Modificadores del group by (with rollup - with cube)

Podemos combinar **group by** con los operadores `with` **rollup** y `with` **cube**.

`with` **rollup** resume los valores por grupo.

```
select ciudad, count(*) as cantidad
from visitantes
group by ciudad with rollup;
```

La consulta anterior retorna los registros agrupados por ciudad y una fila extra en la que la primera columna contiene **null** y la columna con la cantidad muestra la cantidad total. Tambien se pueden hacer consultas con comandos de agrupamiento

```
select ciudad,sexo,count(*) as cantidad
from visitantes
group by ciudad,sexo
with rollup;
```

La salida muestra los totales por ciudad y sexo y produce tantas filas extras como valores existen del primer campo por el que se agrupa ("ciudad" en este caso), mostrando los totales para cada valor, con la columna correspondiente al segundo campo por el que se agrupa ("sexo" en este ejemplo) conteniendo "null", y 1 fila extra mostrando el total de todos los visitantes (con las columnas correspondientes a ambos campos conteniendo "null"). Es decir, por cada agrupación, aparece una fila extra con el/ los campos que no se consideran, seteados a "null".

Results | Messages

	ciudad	sexo	cantidad	total
1	Alta Gracia	f	3	413.50
2	Alta Gracia	m	1	25.00
3	Alta Gracia	NULL	4	438.50
4	Cordoba	f	3	115.90
5	Cordoba	m	2	48.50
6	Cordoba	NULL	5	164.40
7	Villa Dolores	f	1	95.00
8	Villa Dolores	NULL	1	95.00
9	NULL	NULL	10	697.90

Figura 1: Tabla con **rollup**

4.9. with cube

Similar al comando `with` **rollup**, `with` **cube** permite genera filas de resumen de subgrupos para todas las combinaciones posibles de los valores de los campos por los que agrupamos. Se pueden colocar hasta 10 campos en el **group by**. Con **cube** se puede emplear **where** y**having**, pero no es compatible con **all**.

```
select sexo,estadocivil,seccion,
count(*) from empleados
group by sexo,estadocivil,seccion
with cube;
```

Results | Messages

	sexo	estadocivil	seccion	(No column name)
1	f	c	Administracion	4
2	m	c	Administracion	1
3	NULL	c	Administracion	5
4	f	s	Administracion	1
5	m	s	Administracion	1
6	NULL	s	Administracion	2
7	f	v	Administracion	1
8	m	v	Administracion	1
9	NULL	v	Administracion	2
10	NULL	NULL	Administracion	9
11	f	c	Sistemas	2
12	m	c	Sistemas	3
13	NULL	c	Sistemas	5
14	m	s	Sistemas	3
15	NULL	s	Sistemas	3
16	NULL	NULL	Sistemas	8
17	NULL	NULL	NULL	17
18	f	NULL	Administracion	6
19	f	NULL	Sistemas	2
20	f	NULL	NULL	8
21	m	NULL	Administracion	3
22	m	NULL	Sistemas	6
23	m	NULL	NULL	9
24	f	c	NULL	6

Figura 2: Tabla con **cube**.

4.10. grouping

La función `grouping` se emplea con los operadores **rollup** y **cube** para distinguir los valores de detalle y de resumen en el resultado. Es decir, permite diferenciar si los valores **null** que aparecen en el resultado son valores nulos puramente de las tablas o si son una fila generada por los operadores **rollup** o **cube**.

Con esta función aparece una nueva columna en la salida, una por cada `grouping`; retorna el valor 1 para indicar que la fila representa los valores de resumen de **rollup** o **cube** y el valor 0 para representar los valores de campo.

Sólo se puede emplear la función `grouping` en los campos que aparecen en la cláusula **`group by`**. Entonces, si emplea los operadores **`rollup`** y **`cube`** y los campos por los cuales agrupa admiten valores nulos, utilice la función `grouping` para distinguir los valores de detalle y de resumen en el resultado.

```
select ciudad,
count(*) as cantidad,
grouping(ciudad) as resumen
from visitantes
group by ciudad
with rollup;
```

Results | Messages

	ciudad	cantidad	resumen
1	NULL	1	0
2	Alta Gracia	1	0
3	Cordoba	3	0
4	NULL	5	1

Figura 3: Tabla con `grouping`.

4.11. Registros duplicados (distinct)

Con la cláusula **`distinct`** podemos obviar registros que se encuentren duplicados a la hora de hacer una consulta.

Esta puede combinarse con codigos del estilo **`group by`**, **`order by`** y **`where`**.

```
select distinct autor
from libros;

select count(distinct autor)
from libros;
```

4.12. Primeros registros (top - with ties)

La palabra clave `top` se emplea para obtener sólo una cantidad limitada de registros, los primeros n registros de una consulta.

```
select top 3 autor
from libros;

--Puede combinarse con order by
select top 3 autor
from libros
group by editorial;
```

Al utilizar **order by** uno puede combinarlo con el comando `with ties`. En tal caso, este comando muestra todos los registros con los cuales se agrupó.

```
--Con top y with ties
select top 3 with ties
* from libros
order by autor;
```

	titulo	autor
1	El aleph	Borges
2	Alicia en el pais...	Carroll
3	Aprenda PHP	Mario Molina
4	Java en 10 minutos	Mario Molina
5	Java desde cero	Mario Molina
6	Ilusiones	Richard Bach
7	Uno	Richard Bach

	titulo	autor
1	El aleph	Borges
2	Alicia en el pais...	Carroll
3	Aprenda PHP	Mario Molina
4	Java en 10 minutos	Mario Molina
5	Java desde cero	Mario Molina

Figura 4: **Izq:** tenemos la tabla sin ninguna cláusula. **Der:** utilizamos la combinación de `top` con `with ties`

Otro argumento para cuando usamos `top` es el código `percent` indicando el porcentaje de registros a recuperar y no la cantidad. Hay que tener en cuenta que los valores fraccionarios se redondean al siguiente valor entero.

```
--Seleccionamos el 50% de la tabla
select top 50 percent *
from libros
order by autor;
```

4.13. Lenguaje de control de flujo (case)

Sirve para comparar 2 o mas valores (o casos) devolviendo un resultado. La sintaxis es:

```
case VALORACOMPARAR
when VALOR1 then RESULTADO1
when VALOR2 then RESULTADO2
...
else RESULTADO3
end
```

Por cada valor hay un **when** y un **then**; si encuentra un valor coincidente en algún **when** ejecuta el **then** correspondiente a ese **when**, si no encuentra ninguna coincidencia, se ejecuta el **else**; si no hay parte **else** retorna **null**. Finalmente se coloca **end** para indicar que el **case** ha finalizado.

Podemos usar distintos comparadores en la sentencia **case**, por ejemplo:

```
select nombre, nota, condicion=
case
when nota = 1 then 'ausente o burro'
when nota<4 then 'libre'
when nota >=4 and nota<7 then 'regular'
when nota>=7 then 'promocionado'
else 'sin nota'
end
from alumnos;
```

Puede utilizar una expresión **case** en cualquier lugar en el que pueda utilizar una expresión.

También se puede emplear con **group by** y funciones de agrupamiento.

4.14. Lenguaje de control de flujo II (if)

if y **else** testean una condición; se emplean cuando un bloque de sentencias debe ser ejecutado si una condición se cumple y si no se cumple, se debe ejecutar otro bloque de sentencias diferente.

```
if exists (select * from libros where cantidad=0)
(select titulo from libros where cantidad=0)
else
select 'No hay libros sin stock';
```

Podemos emplear "**if** ... **else**" en actualizaciones. Por ejemplo, queremos hacer un descuento en el precio, del 10 % a todos los libros de una determinada editorial; si no hay, mostrar un mensaje:

```
if exists (select * from libros where editorial='Emece')
begin
update libros set precio=precio-(precio*0.1) where editorial='Emece'
select 'libros actualizados'
end
else
select 'no hay registros actualizados';
```

Note que si la condición es verdadera, se deben ejecutar 2 sentencias. Por lo tanto, se deben encerrar en un bloque "**begin** ... **end**".

Obs: Dentro de los comandos SQL select, update y delete no podemos hacer uso de la sentencia de control de flujo if, debemos utilizar la sentencia case que vimos en el concepto anterior. A partir de la versión 2012 de SQL Server disponemos de la función integrada "iif".

5. Restricciones y reglas

Muchas de las funciones de esta sección van a ser muy similares que las de los capítulos anteriores. La diferencia en este capitulo es que vamos a trabajar con tablas que ya estan creadas y a partir de ahi vamos a darles restricciones a los campos.

5.1. Información sobre restricciones (sp_helpconstraint)

Para obtener información sobre las distintas restricciones que hay en las tablas

```
exec sp_helpconstraint TABLA;
```

5.2. default

La restricción **default** permite darle un valor por defecto para un campo cuando no se inserta explicitamente en un comando **insert**.

Cada vez que establecíamos un valor por defecto para un campo de una tabla, SQL Server creaba automáticamente una restricción **default** para ese campo de esa tabla.

Dicha restricción, a la cual no le dábamos un nombre, recibía un nombre dado por SQL Server que consiste "DF" (por default), seguido del nombre de la tabla, el nombre del campo y letras y números aleatorios.

Podemos agregar una restricción **default** a una tabla existente con la sintaxis básica siguiente:

```
alter table NOMBRETABLA
add constraint NOMBRECONSTRAINT
default VALORPORDEFECTO
for CAMPO;
```

Por convención, cuando demos el nombre a las restricciones **default** emplearemos un formato similar al que le da SQL Server: "DF_NOMBRETABLA_NOMBRECAMPO".

5.3. check

La restricción **check** especifica los valores que acepta un campo, evitando que se ingresen valores inapropiados.

Si en una tabla queremos insertar los precios de libros por minorista y mayorista como tipo **decimal**(5,2), estos deben ser positivos siempre. Para hacer esto imponemos una condición directamente en la tabla (no en la consulta).

```
--Caso general
alter table NOMBRETABLA
add constraint NOMBRECONSTRAINT
check CONDICION;

alter table libros
add constraint CK_libros_precio_positivo
check (preciomin>=0 and preciomay>=0);
```

Este tipo de restricción verifica los datos cada vez que se ejecuta una sentencia **insert** o **update**, es decir, actúa en inserciones y actualizaciones.

Si la tabla contiene registros que no cumplen con la restricción que se va a establecer, la restricción no se puede establecer, hasta que todos los registros cumplan con dicha restricción.

No se puede aplicar esta restricción junto con la propiedad **identity**.

5.4. Deshabilitar restricciones (with check - nocheck)

Dijimos que podemos utilizar la restricción **check** una vez que los campos existentes cumplan la condición que vamos a imponer. Para introducir la condición **check** igualmente sin importar los datos anteriores usamos la restricción `with nocheck`

```
alter table libros
with nocheck
add constraint CK_libros_precio
check (precio>=0);

--O deshabilitar la condicion en la tabla
alter table libros
nocheck constraint CK_libros_precio;
```

Para habilitar o deshabilitar restricciones la comprobación de datos en inserciones o actualizaciones, la sintaxis básica es:

```
alter table NOMBRETABLA
OPCIONdeRESTRICCION constraint NOMBRERESTRICCION;
```

5.5. Restricción primary key

Podemos agregar una restricción "primary key" a una tabla existente con la sintaxis básica siguiente:

```
alter table NOMBRETABLA
add constraint NOMBRECONSTRAINT
primary key(NOMBRECAMPO,...);
```

Cumple la misma función que cuando creabamos una tabla, la diferencia es que ahora estamos modificando una tabla ya existente.

5.6. unique

Similar a la restricción **primary key**, el comando **unique** impide duplicar datos de un campo que no sean primarios (a excepcion de los valores **null**). Se pueden aplicar varias restricciones de este tipo a una misma tabla.

Se emplea cuando ya se estableció una clave primaria (como un número de legajo) pero se necesita asegurar que otros datos también sean únicos y no se repitan (como número de documento).

```
--General
alter table NOMBRETABLA
```

```
add constraint UQ_NOMBRETABLA_NOMBRECAMPO
unique (CAMPOS);

--Ejemplo
alter table alumnos
add constraint UQ_alumnos_documento
unique (documento);
```

5.7. Elimiar restricciones (drop)

```
alter table NOMBRETABLA
drop NOMBRERESTRICCION;
```

5.8. Crear y asociar reglas (create rule - sp_bindrule)

Las reglas especifican los valores que se pueden ingresar en un campo, asegurando que los datos se encuentren en un intervalo de valores específico, coincidan con una lista de valores o sigan un patrón. Una regla se asocia a un campo de una tabla (o a un tipo de dato definido por el usuario, tema que veremos posteriormente).

Un campo puede tener solamente UNA regla asociado a él. La sintaxis básica para crear reglas es:

```
create rule NOMBREREGLA
as @VARIABLE CONDICION;

--Ejemplo
create rule RG_sueldo_intervalo
as @sueldo between 100 and 1000
```

La variable puede tener cualquier nombre, pero debe estar precedido por el signo arroba (@), dicha variable será reemplazada por el valor del campo cuando se asocie.

La condición se refiere a los valores permitidos para inserciones y actualizaciones y puede contener cualquier expresión válida para una cláusula **where**; no puede hacer referencia a los campos de una tabla.

Luego de crear la regla, debemos asociarla a un campo ejecutando un procedimiento almacenado del sistema empleando la siguiente sintaxis básica:

```
exec sp_bindrule NOMBREREGLA, 'TABLA.CAMPO';

--Ejemplo
exec sp_bindrule RG_sueldo_intervalo, 'empleados.sueldo';
```

La función que cumple una regla es básicamente la misma que una restricción **check**, las siguientes características explican algunas diferencias entre ellas:

- podemos definir varias restricciones **check** sobre un campo, un campo solamente puede tener una regla asociada a él;
- una restricción **check** se almacena con la tabla, cuando ésta se elimina, las restricciones también se borran. Las reglas son objetos diferentes e independientes de las tablas, si eliminamos una tabla, las asociaciones desaparecen, pero las reglas siguen existiendo en la base de datos;
- una restricción **check** puede incluir varios campos; una regla puede asociarse a distintos campos (incluso de distintas tablas);
- una restricción **check** puede hacer referencia a otros campos de la misma tabla, una regla no.

5.9. Valores predeterminados (create default - sp_bindefault)

Los valores predeterminados se asocian con uno o varios campos (o tipos de datos definidos por el usuario); se definen una sola vez y se pueden usar muchas veces.

Si no se coloca un valor cuando se ingresan datos, el valor predeterminado especifica el valor del campo al que está asociado. La sintaxis básica:

```
create default NOMBREVALORPREDETERMINADO
as VALORPREDETERMINADO;

--Ejemplo
create default VP_datodesconocido
as 'Desconocido'
```

"VALORPREDETERMINADO" no puede hacer referencia a campos de una tabla (u otros objetos) y debe ser compatible con el tipo de datos y longitud del campo al cual se asocia.

Luego de crear un valor predeterminado debemos asociarlo a un campo (o a un tipo de dato definido por el usuario):

```
exec sp_bindefault NOMBRE, 'NOMBRETABLA.CAMPO';

--Ejemplo
exec sp_bindefault VP_datodesconocido, 'empleados.domicilio';
```

Podemos asociar un valor predeterminado a varios campos.

La función que cumple un valor predeterminado es básicamente la misma que una restricción **default**; las siguientes características explican algunas semejanzas y diferencias entre ellas:

- Un campo solamente puede tener definida UNA restricción **default**, un campo solamente puede tener UN valor predeterminado asociado a él.
- Una restricción **default** se almacena con la tabla; cuando ésta se elimina, las restricciones también. Los valores predeterminados son objetos diferentes e independientes de las tablas: si eliminamos una tabla, las asociaciones desaparecen, pero los valores predeterminados siguen existiendo en la base de datos.
- Una restricción **default** se establece para un solo campo; un valor predeterminado puede asociarse a distintos campos (inclusive, de diferentes tablas).
- Una restricción **default** no puede establecerse sobre un campo **identity**, tampoco un valor predeterminado.

No se puede asociar un valor predeterminado a un campo que tiene una restricción **default**.

5.10. Desasociar y eliminar valores predeterminados (sp_unbindefault - drop)

Un valor predeterminado no puede eliminarse si no se ha desasociado previamente.

Para deshacer una asociación empleamos el procedimiento almacenado **sp_unbindefault** seguido de la tabla y campo al que está asociado:

```
exec sp_unbindefault 'TABLA.CAMPO';

--Ejemplo
exec sp_unbindefault  'empleados.sueldo';
```

Luego, con la instrucción **drop default** podemos eliminar un valor predeterminado:

```
drop default NOMBREVALORPREDETERMINADO

--Ejemplo
drop default VP_cero;
```

5.11. Información sobre reglas (sp_help - sp_helpconstraint)

Podemos utilizar el procedimiento almacenado **sp_help** con el nombre del objeto del cual queremos información, en este caso el nombre de una regla. muestra nombre, propietario, tipo y fecha de creación:

Con **sp_help**, no sabemos si las reglas existentes están o no asociadas a algún campo

```
exec sp_help NOMBREREGLA;
```

sp_helpconstraint retorna una lista de todas las restricciones que tiene una tabla. Podemos ver las reglas asociadas a una tabla con este procedimiento almacenado:

```
exec sp_helpconstraint NOMBRETABLA;
```

Muestra la siguiente información:

Muestra la siguiente información:

- `constraint_type`: indica que es una regla con `rule`, nombrando el campo al que está asociada.
- `constraint_name`: nombre de la regla.
- `constraint_keys`: muestra el texto de la regla.

Para ver el texto de una regla empleamos el procedimiento almacenado "sp_helptext"seguido del nombre de la regla:

```
exec sp_helptext NOMBREREGLA;
```

También se puede consultar la tabla del sistema `sysobjects`, que nos muestra el nombre y varios datos de todos los objetos de la base de datos actual. La columna "xtype" indica el tipo de objeto, en caso de ser una regla aparece el valor "R":

```
select * from sysobjects;;
```

6. índices

El objetivo de un indice es acelerar la recuperación de información. La indexación es una técnica que optimiza el acceso a los datos, mejora el rendimiento acelerando las consultas y otras operaciones. Es útil cuando la tabla contiene miles de registros, cuando se realizan operaciones de ordenamiento y agrupamiento y cuando se combinan varias tablas (tema que veremos más adelante).
La desventaja es que consume espacio en el disco y genera costo de mantenimiento (tiempo y recursos).

Los índices más adecuados son aquellos creados con campos que contienen valores únicos.

Existen dos tipos de índices: 1) agrupados (clustered) y 2) no agrupados (nonclustered).

1) Un INDICE AGRUPADO es similar a una guía telefónica, los registros con el mismo valor de campo se agrupan juntos. Un índice agrupado determina la secuencia de almacenamiento de los registros en una tabla. Se utilizan para campos por los que se realizan búsquedas con frecuencia o se accede siguiendo un orden. Una tabla sólo puede tener UN índice agrupado. El tamaño medio de un índice agrupado es aproximadamente el 5 % del tamaño de la tabla.

2) Un INDICE NO AGRUPADO es como el índice de un libro, los datos se almacenan en un lugar diferente al del índice, los punteros indican el lugar de almacenamiento de los elementos indexados en la tabla.
Un índice no agrupado se emplea cuando se realizan distintos tipos de busquedas frecuentemente, con campos en los que los datos son únicos.

6.1. Crear índices

La sintaxis básica es:

```
create TIPOINDICE index NOMBREINDICE
on TABLA(CAMPO 1 ,...)
```

"TIPODEINDICE" indica si es agrupado (`clustered`) o no agrupado (`nonclustered`). Si no especificamos crea uno No agrupado. Independientemente de si es agrupado o no, también se puede especificar que sea **`unique`**, es decir, no haya valores repetidos. Si se intenta crear un índice **`unique`** para un campo que tiene valores duplicados, SQL Server no lo permite.

Un índice puede tener más de un campo como clave, son índices compuestos. Los campos de un índice compuesto tienen que ser de la misma tabla (excepto cuando se crea en una vista - tema).

Para ver información sobre los indices creados podemos usar tanto **`sp_helpconstraint`** (para ver si es `clusteres` o `nonclustered`) o **`sp_helpindex`** para ver los indices de una tabla.

```
exec   sp_helpindex      libros;
```

6.2. Regenerar índices

Empleando la opción `drop_existing` junto con **`create index`** permite regenerar un índice, con ello evitamos eliminarlo y volver a crearlo. La sintaxis es la siguiente:

```
create TIPODEINDICE index NOMBREINDICE
on TABLA(CAMPO)
with drop_existing;
```

También podemos modificar alguna de las características de un índice con esta opción, a saber:

- **tipo**: cambiándolo de **no agrupado** a **agrupado** (siempre que no exista uno agrupado para la misma tabla). No se puede convertir un índice agrupado en No agrupado.
- **campo**: se puede cambiar el campo por el cual se indexa, agregar campos, eliminar algún campo de un **índice compuesto**.
- **único**: se puede modificar un índice para que los valores sean **únicos** o dejen de serlo.

```sql
--Ejemplo
--Creamos el indice
create nonclustered index I_libros
on libros(titulo);

--Lo regeneramos con clustered ahora
create clustered index I_libros
on libros(titulo)
with drop_existing;

--Le agregamos un campo extra
create clustered index I_libros
on libros(titulo,editorial)
with drop_existing;
```

6.3. Eliminar índices

Los índices creados con **create index** se eliminan con **drop index**; la siguiente es la sintaxis básica:

```sql
drop index NOMBRETABLA.NOMBREINDICE;
```

Los índices que SQL Server crea automáticamente al establecer una restricción **primary key** o **unique** no pueden eliminarse con **drop index**, se eliminan automáticamente cuando quitamos la restricción.

Podemos averiguar si existe un índice para eliminarlo, consultando la tabla del sistema `sysindexes` :

```sql
if exists (select name from sysindexes
  where name = 'NOMBREINDICE')
  drop index NOMBRETABLA.NOMBREINDICE;
```

7. Combinaciones

7.1. Combinaciones básicas (join - left/right join - full join)

En muchisimos casos es habitual trabajar con varias tablas que tienen uno o más campos relacionados entre si. Si uno quiere realizar alguna consulta que involucre campos de ambas tablas es necesario utilizar los distintos comandos de **join**.
Lo que uno realiza con estos comandos es una forma de combinar información de una tabla con otra y pueden ser mejor explicados con una imagen:

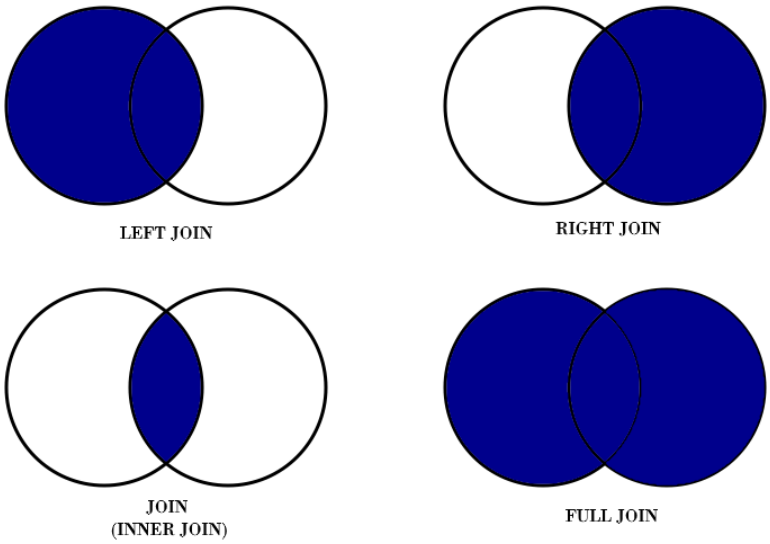

Figura 5:

- **LEFT JOIN**: Devuelve *todas* las filas de la tabla izquierda y las coincidencias de la tabla derecha (rellena con NULL si no hay coincidencia).

```
select titulo,nombre
from libros as l
left join editoriales as e
on codigoeditorial = e.codigo;
```

- **RIGHT JOIN**: Devuelve *todas* las filas de la tabla derecha y las coincidencias de la tabla izquierda (rellena con NULL si no hay coincidencia).

```
select titulo,nombre
from editoriales as e
left join libros as l
on codigoeditorial = e.codigo;
```

- **JOIN (INNER JOIN)**: Devuelve únicamente las filas donde existe una coincidencia en *ambas* tablas.

```
  select l.codigo,titulo,autor,nombre
  from libros as l
  join editoriales as e
  on l.codigoeditorial=e.codigo;
```

- **FULL JOIN**: Devuelve todas las filas siempre que haya una coincidencia en *cualquiera* de las tablas (es la combinación de Left y Right Join).

```
select titulo,nombre
from editoriales as e
full join libros as l
on codigoeditorial = e.codigo;
```

En varios casos el uso de alias es para fines de simplificación y hace más legible la consulta si es larga y compleja, pero en algunas consultas es absolutamente necesario.
Se puede incluir en la consulta join la cláusula **where** para restringir los registros que retorna el resultado; también **order by**, **distinct**, etc..

7.2. Combinación cruzada (cross join)

Muestran todas las combinaciones de todos los registros de las tablas combinadas. Para este tipo de join no se incluye una condición de enlace. Se genera el producto cartesiano en el que el número de filas del resultado es igual al número de registros de la primera tabla multiplicado por el número de registros de la segunda tabla, es decir, si hay 5 registros en una tabla y 6 en la otra, retorna 30 filas.

```
--Caso general
select CAMPOS
from TABLA1
cross join TABLA2;

--Ejemplo
select c.nombre as 'plato principal', p.nombre as 'postre'
from comidas as c
cross join postres as p;
```

7.3. Autocombinación

En una autocombinación se combina una tabla con una copia de si misma. Para ello debemos utilizar 2 alias para la tabla. Para evitar que aparezcan filas duplicadas, debemos emplear un **where**.

```
select c1.nombre as 'plato principal',
c2.nombre as postre,
c1.precio+c2.precio as total
from comidas as c1
join comidas as c2
on c1.codigo<>c2.codigo
where c1.rubro='plato' and
c2.rubro='postre';
```

7.4. Combinaciones con update - delete

```
--Ejemplo con update
 update libros set precio=precio+(precio*0.1)
from libros
join editoriales as e
on codigoeditorial=e.codigo
where nombre='Planeta';

--Ejemplo con delete
delete libros
from libros
join editoriales
on codigoeditorial = editoriales.codigo
where editoriales.nombre='Emece';
```

7.5. Combinación de más de dos tablas

Cada vez que realizamos un join es siempre entre dos tablas, por lo que es posible combinar varias tablas a partir de utilizar varios **join**

```
select titulo,a.nombre,e.nombre
from autores as a
join libros as l
on codigoautor=a.codigo
join editoriales as e  on codigoeditorial=e.codigo;
```

8. Clave foránea (foreign key) y restricciones

Un campo que no es clave primaria en una tabla y sirve para enlazar sus valores con otra tabla en la cual es clave primaria se denomina clave foránea, externa o ajena.

En el ejemplo de la librería en que utilizamos las tablas "librosz .^{ed}itorialesçon estos campos:

libros: codigo (clave primaria), titulo, autor, codigoeditorial, precio y editoriales: codigo (clave primaria), nombre.

El campo çodigoeditorial"de "libros.^{es} una clave foránea, se emplea para enlazar la tabla "librosçon .^{ed}itorialesz es clave primaria en .^{ed}itorialesçon el nombre çodigo".

La siguiente es la sintaxis parcial general para agregar una restricción "foreign key":

```
alter table NOMBRETABLA1
add constraint NOMBRERESTRICCION
foreign key (CAMPOCLAVEFORANEA)
references NOMBRETABLA2 (CAMPOCLAVEPRIMARIA);
```

```
5
6  --Ejemplo
7  alter table libros
8  add constraint FK_libros_codigoeditorial
9  foreign key (codigoeditorial)
10 references editoriales(codigo);
```

En el ejemplo implementamos una restricción "foreign key" para asegurarnos que el código de la editorial de la tabla "libros" ("codigoeditorial") esté asociada con un código válido en la tabla "editoriales" ("codigo").

Cuando agregamos cualquier restricción a una tabla que contiene información, SQL Server controla los datos existentes para confirmar que cumplen con la restricción, si no los cumple, la restricción no se aplica y aparece un mensaje de error. Por ejemplo, si intentamos agregar una restricción "foreign key" a la tabla "libros" y existe un libro con un valor de código para editorial que no existe en la tabla "editoriales", la restricción no se agrega.

Actúa en inserciones. Si intentamos ingresar un registro (un libro) con un valor de clave foránea (codigoeditorial) que no existe en la tabla referenciada (editoriales), SQL server muestra un mensaje de error. Si al ingresar un registro (un libro), no colocamos el valor para el campo clave foránea (codigoeditorial), almacenará **null**, porque esta restricción permite valores nulos (a menos que se haya especificado lo contrario al definir el campo).

Actúa en eliminaciones y actualizaciones. Si intentamos eliminar un registro o modificar un valor de clave primaria de una tabla si una clave foránea hace referencia a dicho registro, SQL Server no lo permite (excepto si se permite la acción en cascada, tema que veremos posteriormente). Por ejemplo, si intentamos eliminar una editorial a la que se hace referencia en "libros", aparece un mensaje de error.

Una tabla puede tener varias restricciones "foreign key".

No se puede eliminar una tabla referenciada en una restricción "foreign key", aparece un mensaje de error.

Una restriccion "foreign key" no puede modificarse, debe eliminarse y volverse a crear.

8.1. Restricciones foreign key en la misma tabla

Veamos un ejemplo en el cual definimos esta restricción dentro de la misma tabla: Una mutual almacena los datos de sus afiliados en una tabla llamada "afiliados". Algunos afiliados inscriben a sus familiares. La tabla contiene un campo que hace referencia al afiliado que lo incorporó a la mutual

En caso que un afiliado no haya sido incorporado a la mutual por otro afiliado, el campo "afiliadotitular" almacenará **null**. Establecemos una restricción "foreign key" para asegurarnos que el número de afiliado que se ingrese en el campo "afiliadotitular" exista en la tabla "afiliados".
La sintaxis es la misma, excepto que la tabla se autoreferencia. Luego de aplicar esta restricción, cada vez que se ingrese un valor en el campo "afiliadotitular", SQL Server controlará que dicho número exista en la tabla, si no existe, mostrará un mensaje de error.

	numero	documento	nombre	afiliadotitular
1	1	22222222	Perez Juan	NULL
2	2	23333333	Garcia Maria	NULL
3	3	24444444	Lopez Susana	NULL
4	4	30000000	Perez Marcela	1
5	5	31111111	Morales Luis	1
6	6	32222222	Garcia Maria	2

Figura 6: Tabla con una FK autoreferenciada.

```
alter table afiliados
add constraint FK_afiliados_afiliadotitular
foreign key (afiliadotitular)
references afiliados (numero);
```

8.2. Acciones

La restricción **foreign key** tiene las cláusulas **on delete** y **on update** que son opcionales. Estas cláusulas especifican cómo debe actuar SQL Server frente a eliminaciones y modificaciones de las tablas referenciadas en la restricción. Las opciones para estas cláusulas son las siguientes:

- **no action**: indica que si intentamos eliminar o actualizar un valor de la clave primaria de la tabla referenciada (TABLA2) que tengan referencia en la tabla principal (TABLA1), se genere un error y la acción no se realice; es la opción predeterminada.
- **cascade**: indica que si eliminamos o actualizamos un valor de la clave primaria en la tabla referenciada (TABLA2), los registros coincidentes en la tabla principal (TABLA1), también se eliminen o modifiquen; es decir, si eliminamos o modificamos un valor de campo definido con una restricción **primary key** o **unique**, dicho cambio se extiende al valor de clave externa de la otra tabla (integridad referencial en cascada).

```
alter table TABLA1
add constraint NOMBRERESTRICCION
foreign key (CAMPOCLAVEFORANEA)
references TABLA2(CAMPOCLAVEPRIMARIA)
on delete OPCION
on update OPCION;

--En OPCION va 'no action' o 'cascade'
```

9. Subconsultas

Una subconsulta o subquery es una consulta dentro de otra consulta **select**, **insert**, **update** o **delete** (o en otra subconsuta).

Las subconsultas se emplean cuando una consulta es muy compleja, entonces se la divide en varios pasos lógicos y se obtiene el resultado con una única instrucción y cuando la consulta depende de los resultados de otra consulta.
Generalmente, una subconsulta se puede reemplazar por combinaciones y estas últimas son más eficientes. Además, puede haber subconsultas dentro de subconsultas, se admiten hasta 32 niveles de anidación. Las subconsultas se DEBEN incluir entre paréntesis.

Hay tres tipos básicos de subconsultas:

(I) Las que retornan un solo valor escalar que se utiliza con un operador de comparación o en lugar de una expresión.

(II) Las que retornan una lista de valores, se combinan con **in**, o los operadores **any**, **some** y **all**.

(III) Las que testean la existencia con **exists**.

9.1. Subconsulta como valor único

Las subconsultas que retornan un solo valor escalar se utilizan con un operador de comparación o en lugar de una expresión:

```
--Casos generales
select CAMPOS
from TABLA
where CAMPO OPERADOR (SUBCONSULTA);

select CAMPO OPERADOR (SUBCONSULTA)
from TABLA;

--Queremos el libro con el mayor precio
select titulo,autor, precio
from libros
where precio=
(select max(precio) from libros);
```

No olvide que las subconsultas luego de un operador de comparación (que no es seguido por **any** o **all**) no pueden incluir cláusulas **group by**.

9.2. Subconsulta con in

El resultado de una subconsulta con **in** (o **not in**) es una lista. Luego que la subconsulta retorna resultados, la consulta exterior los usa.

Este ejemplo muestra los nombres de las editoriales que ha publicado libros de un determinado autor:

```
select nombre
from editoriales
where codigo in
(select codigoeditorial
from libros
where autor='Richard Bach');
```

La subconsulta (consulta interna) retorna una lista de valores de un solo campo (codigo) que la consulta exterior luego emplea al recuperar los datos. Es posible realizar la misma consulta con un **join**:

```
select distinct nombre
from editoriales as e
join libros
on codigoeditorial=e.codigo
where autor='Richard Bach';
```

Una combinación (join) siempre puede ser expresada como una subconsulta, pero una subconsulta no siempre puede reemplazarse por una combinación que retorne el mismo resultado. Si es posible, es aconsejable emplear combinaciones en lugar de subconsultas, son más eficientes.

9.3. any - some - all

any y **some** son sinónimos. ambos chequean si alguna fila de la lista que resulta de la subconsulta se encuentra en el valor especificado de la condición.

```
select titulo
from libros
where autor='Borges' and
codigoeditorial = any
(select e.codigo
from editoriales as e
join libros as l
on codigoeditorial=e.codigo
where l.autor='Richard Bach');
```

La consulta interna (subconsulta) retorna una lista de valores de un solo campo (puede ejecutar la subconsulta como una consulta para probarla), luego la consulta externa compara cada valor de "codigoeditorial" con cada valor de la lista devolviendo los títulos de "Borges" que coinciden.

all también compara un valor escalar con una serie de valores. Chequea si **TODOS** los valores de la lista de la consulta externa se encuentran en la lista de valores devuelta por la consulta interna.

En resumen:

- Emplear = **any** es lo mismo que emplear **in**.
- Emplear <>**all**es lo mismo que emplear **not in**.

Recuerde que solamente las subconsultas luego de un operador de comparación al cual es seguido por any o all pueden incluir cláusulas group by.

9.4. Subconsultas correlacionadas

```
select f.*,
(select count(d.numeroitem)
from Detalles as d
where f.numero=d.numerofactura) as cantidad,
(select sum(d.preciounitario*cantidad)
from Detalles as d
where f.numero=d.numerofactura) as total
from facturas as f;
```

A este tipo de subconsulta se la denomina consulta correlacionada. La consulta interna se evalúa tantas veces como registros tiene la consulta externa, se realiza la subconsulta para cada registro de la consulta externa. El campo de la tabla dentro de la subconsulta (f.numero) se compara con el campo de la tabla externa.

9.5. Subconsultas con exist - not exist

Los operadores **exists** y **not exists** se emplean para determinar si hay o no datos en una lista de valores.

Estos operadores pueden emplearse con subconsultas correlacionadas para restringir el resultado de una consulta exterior a los registros que cumplen la subconsulta (consulta interior). Estos operadores retornan **true** (si las subconsultas retornan registros) o **false** (si las subconsultas no retornan registros).

```
select cliente,numero
from facturas as f
where exists
(select * from Detalles as d
where f.numero=d.numerofactura
and d.articulo='lapiz');
```

Puede hacerse o mismo con un join. El caso **not** `exist` es similar.

9.6. Simil autocombinación

En algunas circunstancias es posible hacer una consultainterna y externa sobre la misma tabla. La idea es muy similar a cuando se utiliza un **join**.

```
--Con subconsulta
select distinct l1.titulo
from libros as l1
where l1.titulo in
(select l2.titulo
from libros as l2
where l1.editorial <> l2.editorial);

--Con join
select distinct l1.titulo
from libros as l1
join libros as l2
on l1.titulo=l2.titulo and
l1.autor=l2.autor
where l1.editorial<>l2.editorial;
```

9.7. Subconsulta como tabla

En algunas situaciones resulta útil trabajar con parte de una (o varias) tablas. Para esto utilizamos subconsultas antes o después de una sentencia **from**:

```
--Antes del from
select f.*,
(select sum(d.precio*cantidad)
from Detalles as d
where f.numero=d.numerofactura) as total
from facturas as f;

--Despues del from
select avg(total)
from (
select sum(d.precio*cantidad) as total
from Detalles as d
where d.precio > 100
)
as t;
```

9.8. Subconsulta con update - delete - insert

```
--UPDATE
update libros set precio=precio+(precio*0.1)
```

```sql
where codigoeditorial=
(select codigo
from editoriales
where nombre='Emece');

--DELETE
delete from libros
where codigoeditorial in
(select e.codigo
from editoriales as e
join libros
on codigoeditorial=e.codigo
where autor='Juan Perez');

--INSERT
-- insertar libros basados en editoriales que tienen mas de 10 libros
insert into libros_archivo (titulo, autor, precio, editorial_nombre)
select titulo, autor, precio, nombre
from libros
join editoriales on libros.codigoeditorial = editoriales.codigo
where editoriales.codigo in (
  select codigoeditorial
  from libros
  group by codigoeditorial
  having count(*) > 10
);
```

10. Vistas

10.1. Vistas en Bases de Datos

Una vista es una alternativa para mostrar datos de varias tablas. Una vista es como una tabla virtual que almacena una consulta. Los datos accesibles a través de la vista no están almacenados en la base de datos como un objeto.

Entonces, una vista almacena una consulta como un objeto para utilizarse posteriormente. Las tablas consultadas en una vista se llaman **tablas base**. En general, se puede dar un nombre a cualquier consulta y almacenarla como una vista.

Una vista suele llamarse también **tabla virtual** porque los resultados que retorna y la manera de referenciarlas es la misma que para una tabla.

Las vistas permiten:

- **Ocultar información:** permitiendo el acceso a algunos datos y manteniendo oculto el resto de la información que no se incluye en la vista. El usuario opera con los datos de una vista como si se tratara de una tabla, pudiendo modificar tales datos.
- **Simplificar la administración de los permisos de usuario:** se pueden dar al usuario

permisos para que solamente pueda acceder a los datos a través de vistas, en lugar de concederle permisos para acceder a ciertos campos, así se protegen las tablas base de cambios en su estructura.

- **Mejorar el rendimiento:** se puede evitar tipear instrucciones repetidamente almacenando en una vista el resultado de una consulta compleja que incluya información de varias tablas.

Podemos crear vistas con:

- un subconjunto de registros y campos de una tabla;
- una unión de varias tablas;
- una combinación de varias tablas;
- un resumen estadístico de una tabla;
- un subconjunto de otra vista, combinación de vistas y tablas.

La sintaxis básica parcial para crear una vista es la siguiente:

```
create view NOMBREVISTA as
SENTENCIAS_SELECT
from TABLA;
```

El contenido de una vista se muestra con un **select**:

```
select * from NOMBREVISTA;
```

Podemos hacer consultas a las vistas como si fueran tablas. Los campos y expresiones de la consulta que define una vista DEBEN tener un nombre, de lo contrario se muestra un error.

10.2. Información sobre vistas

Dado que las vistas son objetos dentro de la base de datos, podemos obtener metadatos e información sobre ellas utilizando procedimientos almacenados del sistema o consultando tablas de sistema directamente.

Existen tres procedimientos principales para inspeccionar vistas:

- **sp_help**: Si se ejecuta sin parámetros, muestra todos los objetos de la base de datos seleccionada (incluidas las vistas). En la columna `Object_type` aparecerá "view" si el objeto es una vista. Si se envía el nombre de una vista como argumento, retorna detalles específicos como la fecha de creación, el propietario y la lista de campos.
- **sp_helptext**: Seguido del nombre de una vista, muestra el código SQL original que la define (la sentencia `CREATE VIEW`), a menos que la vista haya sido encriptada.

- **sp_depends**: Este procedimiento es fundamental para entender las relaciones entre objetos. Al ejecutarlo seguido del nombre de un objeto, obtenemos dos reportes:
  1. Nombre y tipo de los objetos de los cuales *depende* el objeto consultado.
  2. Nombre y tipo de los objetos que *dependen* del objeto consultado.

Si ejecutamos el procedimiento sobre una vista, veremos las **tablas base** referenciadas en ella:

```
exec sp_depends vista_empleados;
```

Por el contrario, si ejecutamos el procedimiento sobre una tabla, veremos qué objetos (como vistas o restricciones) dependen de ella y podrían verse afectados si la modificamos:

```
exec sp_depends empleados;
```

También es posible consultar directamente la tabla del sistema `sysobjects`, la cual contiene metadatos de todos los objetos de la base de datos. La columna `xtype` indica el tipo de objeto; en el caso de las vistas, este valor es **'V'**.

Para ver todos los objetos:

```
select * from sysobjects;
```

Si deseamos filtrar para ver únicamente las vistas creadas por nosotros (por ejemplo, aquellas que comienzan con "vista"), podemos ejecutar:

```
select * from sysobjects
where xtype = 'V'            -- Filtra por tipo 'Vista'
and name like 'vista%';    -- Busqueda con comodin
```

10.3. Encriptado (with encryption)

Podemos ocultar el texto que define una vista empleando la siguiente sintaxis al crearla:

```
create view NOMBREVISTA
with encryption
as
SENTENCIASSELECT
from TABLA;

--Ejemplo
create view vista_empleados
with encryption
as
select (apellido+' '+e.nombre) as nombre,sexo,
```

```
s.nombre as seccion, cantidadhijos
from empleados as e
join secciones as s
on codigo=seccion
```

Si intentamos ejecutar la vista nos va a salir un mensaje de que la vista está encriptada y no nos mostrará la misma.

10.4. Eliminar una vista (drop view)

Si se elimina una tabla a la que hace referencia una vista, la vista no se elimina, hay que eliminarla explícitamente.

Solo el propietario puede eliminar una vista.

La sintaxis es:

```
drop view NOMBREVISTA
```

Antes de eliminar un objeto, se recomienda ejecutar el procedimiento almacenado de sistema **sp_depends** para averiguar si hay objetos que hagan referencia a él.

10.5. Vistas with check option

Es posible obligar a todas las instrucciones de modificación de datos que se ejecutan en una vista a cumplir ciertos criterios.

Por ejemplo, creamos la siguiente vista:

```
create view vista_empleados
as
select apellido, e.nombre, sexo, s.nombre as seccion
from empleados as e
join secciones as s
on seccion=codigo
where s.nombre='Administracion'
with check option;
```

La vista definida anteriormente muestra solamente algunos de los datos de los empleados de la sección "Administracion". Además, solamente se permiten modificaciones a los empleados de esa sección.

Podemos actualizar el nombre, apellido y sexo a través de la vista, pero no el campo "seccion" porque está restringido (ver como se modifican datos a traves de vistas).

10.6. Modificar datos de tablas a traves de vistas

Si se modifican los datos de una vista, se modifica la tabla base.

Se puede insertar, actualizar o eliminar datos de una tabla a través de una vista, teniendo en cuenta lo siguiente, las modificaciones que se realizan a las vistas:

- no pueden afectar a más de una tabla consultada. Pueden modificarse datos de una vista que combina varias tablas pero la modificación solamente debe afectar a una sola tabla.
- no se pueden cambiar los campos resultado de un cálculo.
- pueden generar errores si afectan a campos a las que la vista no hace referencia. Por ejemplo, si se ingresa un registro en una vista que consulta una tabla que tiene campos not null que no están incluidos en la vista.
- la opción "with check option" obliga a todas las instrucciones de modificación que se ejecutan en la vista a cumplir ciertos criterios que se especifican al definir la vista.
- para eliminar datos de una vista solamente UNA tabla puede ser listada en el "from" de la definicion de la misma.

10.7. Modificar vistas (alter view)

Con **alter view** se modifica la definición de una vista sin afectar los procedimientos almacenados y los permisos. Si elimina una vista y vuelve a crearla, debe reasignar los permisos asociados a ella.

En el ejemplo siguiente se altera vista_empleados para agregar el campo "domicilio":

```
alter view vista_empleados
with encryption
as
select (apellido+' '+e.nombre) as nombre,sexo,
s.nombre as seccion, cantidadhijos,domicilio
from empleados as e
join secciones as s
on codigo=seccion
```

Si creó la vista con `with encryption` y quiere modificarla manteniendo la encriptación, debe colocarla nuevamente, en caso de no hacerlo, desaparece.

Operador	Descripción
=	Igual
<>	Distinto
>	Mayor
<	Menor
>=	Mayor o igual
<=	Menor o igual

11. OTROS

11.1. Operadores relacionales

11.2. Comando go

Esto solo se aplica cuando instale el SQL Server en su máquina y utiliza el "SQL Server Management Studio".

El comando `go` sirve para correr un lote de codigo. Se ejecuta todo lo que se encuentre antes del `go` o código entre dos de estos.

Por ejemplo, existen sentencias que necesitan de un comando **`execute`** o puede utilizarse `go`.

```
--MAL
select * from empleados;
sp_helpconstraintempleados;

--BIEN

select * from empleados;
execsp_helpconstraint  empleados;

--Separado en lotes go

select * from empleados;
go
sp_helpconstraint empleados;
```

Las siguientes sentencias no pueden ejecutarse en el mismo lote: **`create`** `rule`, **`create default`**, **`create view`**, **`create`** `procedure` , **`create trigger`**. Cada una de ellas necesita ejecutarse separándolas con `go`.

Referencias

[1] Nombre Apellido. *Título del Libro*. Editorial Universitaria, 2023.

[2] Nombre Apellido and Nombre2 Apellido2. Título del artículo interesante. *Nombre de la Revista*, 15:100–120, 2024.